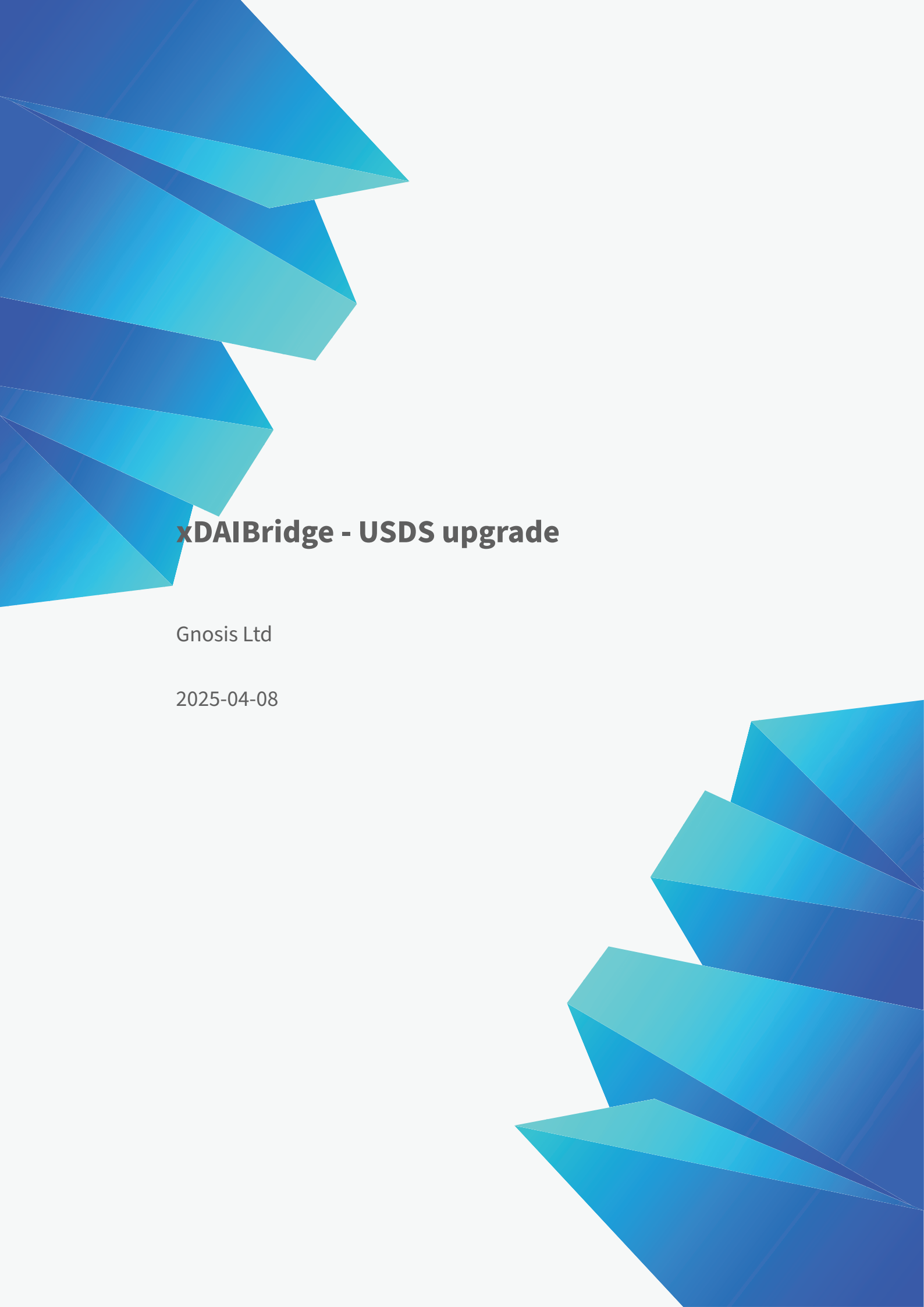




xDAIBridge - USDS upgrade

Gnosis Ltd

2025-04-08

Table of contents

- Table of contents
- xDAIBridge - USDS upgrade Audit
 - Scope
 - Context
 - Status
- Issues
 - [Info] Interacting with the bridge during the upgrade may fail

xDAIBridge - USDS upgrade Audit

This document presents the findings of a smart contract audit conducted for Gnosis Ltd.

Scope

The scope includes contracts modified by PR [gnosischain/tokenbridge-contracts/pull/7](#) the last commit reviewed of the PR being [0x09d73fb](#).

The PR was later reviewed as of commit [0x8af7fb2](#).

Context

The goal is to switch from using and holding DAI/sDAI in the bridge to USDS/sUSDS. To do so, the [XDaiForeignBridge](#) contract that lives on ethereum mainnet is updated to handle USDS and invest USDS into sUSDS instead of handling DAI. A [BridgeRouter](#) helper contract has been implemented to help users bridge different tokens. The router should expose a consistent behaviour before and after the bridge upgrade.

The user interactions with the router for bridging/claiming USDS and DAI before and after the upgrade are described in [tokenbridge-contracts/USDSMigration.md](#).

The newly deployed contracts are:

- [BridgeRouter](#) as a bridge router implementation contract
- [TransparentUpgradeableProxy](#) as a proxy for the bridge router with constructor arguments (`logic=BridgeRouter`, `initialOwner=RouterProxyOwner`, `data=abi.encodeWithSignature("initialize(address)", routerOwner)` to initialize the bridge router with an owner.
- [XDaiForeignBridge](#) as the new bridge implementation contract
- [XDaiBridgePeripheral](#) with the bridge router's address as constructor arguments
- [XDaiBridgePeripheralForDaiPreUsdsUpgrade](#) with the bridge router's address as constructor arguments
- [XDaiBridgePeripheralForUsdsPreUsdsUpgrade](#) with the bridge router's address as constructor arguments

To upgrade the bridge, the following functions must be called:

- `bridgeProxy.upgradeTo(initialVersion + 1, address(XDaiForeignBridgeNewImpl))` by the bridge proxy owner

- `bridgeProxy.swapSDAIToUSDS()`
- `bridgeProxy.initializeInterest(USDS, minCashThreshold, minInterestPaid, gnosisInterestReceiver)` by the bridge owner
- `bridgeProxy.invest(USDS)`
- `BridgeRouter.setRoute(DAI, xDAIBridgeperipheral)` by the router owner
- `BridgeRouter.setRoute(USDS, xDAIForeignBridge)` by the router owner

The values used as constructor arguments or to upgrade the bridge are:

- `BridgeRouter` the address of the newly deployed bridge router implementation contract
- `RouterProxyOwner` is the same as bridge owner `0x42F38ec5A75acCEc50054671233dfAC9C0E7A3F6`
- `routerOwner` is the same as bridge owner `0x42F38ec5A75acCEc50054671233dfAC9C0E7A3F6`
- `initialVersion` = 8
- `XDaiForeignBridgeNewImpl` the address of the new XDaiForeignBridge implementation
- `USDS` = `0xdC035D45d973E3EC169d2276DDab16f1e407384F`
- `minCashThreshold` = `100000000000000000000000000000000`
- `minInterestPaid` = `100000000000000000000000000000000`
- `gnosisInterestReceiver` = `0x670daeaF0F1a5e336090504C68179670B5059088`
- `DAI` = `0x6B175474E89094C44Da98b954EedeAC495271d0F`
- `xDAIBridgePeripheral` the address of the new XDaiBridgePeripheral contract
- `xDAIForeignBridge` = `0x4aa42145Aa6Ebf72e164C9bBC74fbD3788045016`

The deployed bytecode of the contracts can be obtained by running `npm run get-deployed-bytecode` and looking at `scripts/usds_migration/deployBytecode_usds_migration.json` or browsing the `out` directory.

The keccak256 hashes of the contracts should be:

- xDaiForeignBridge: 0x6e7ca817c9686c0ee3dbcd99e9d498d704fd3aae47c1fab98a6f81adaabff95b
- BridgeRouter: 0xc4f7cdc0ec90c22bca996d328c7dc11e20ef11f017eedb8df3d9b6e035f560b4
- XDaiBridgePeripheral: 0x3d5e7ff9da196691d2dc57c93036c7414b68d63921dce530c5747017ea39afb5
- XDaiBridgePeripheralForDaiPreUsdsUpgrade: 0x721cfb942f77f4b04de65a4dbb7b1686e2c363689c83de3bee58da3ab9ae0bb3
- XDaiBridgePeripheralForUsdsPreUsdsUpgrade: 0x74284da7c9b250a0348d431981364bf4e82987ca93524f2c392ea2455fd1e8a9

- TransparentUpgradeableProxy: 0x92579f452fe663595a898cbac85d80bb3868a6c9f034f19ba7fbebdfa3b65a4d

Status

The report has been sent to the core developer.

The PR has been updated to add comments based on the report and the last reviewed commit is now [0x8af7fb2](#).

Issues

[Info] Interacting with the bridge during the upgrade may fail

Summary

After calling `bridgeProxy.upgradeTo(initialVersion + 1, address(XDaiForeignBridgeNewIm))` but before setting the routes on the BridgeRouter or swapping the DAI held by the bridge to USDS, interacting with the bridge through the router may fail.

Vulnerability Detail

- Relaying DAI through `BridgeRouter.relayTokens(DAI, receiver, amount)` will call `XDaiBridgePeripheralForDaiPreUsdsUpgrade.relayTokens()` which approves the bridge to spend DAI before calling `xDAIForeignBridge.relayTokens()`. The bridge will attempt to relay USDS but will fail since the peripheral approved DAI.
- Claiming DAI through `BridgeRouter.executeSignatures()` calls `xDAIForeignBridge.executeSignatures()` where the bridge attempts to convert withdrawn tokens from USDS to DAI before sending them. This will fail if `bridgeProxy.swapSDAIToUSDS()` has not been called and the bridge owns no USDS.
- Relaying USDS through `BridgeRouter.relayTokens(DAI, receiver, amount)` will call `XDaiBridgePeripheralForUsdsPreUsdsUpgrade.relayTokens()` which converts the USDS to DAI and approves the bridge to spend the DAI before calling `xDAIForeignBridge.relayTokens()`. This will fail for the same reason relaying DAI will fail.

Impact

During the upgrade, interaction with the bridge may fail.

Recommendation

Acknowledge the issue

Response

Since the bridge proxy owner, the bridge owner, the router proxy owner, and the router owner are the same address (`0x42F38ec5A75acCEc50054671233dfAC9C0E7A3F6`) all the transactions

necessary for the update can be bundled in a single safe transaction.